

CS3520
Computer Organisation and Architecture I
Lecture 3 – Computer System Overview

Mr K.O, Setetemela
BJO 308
Department of Mathematics & Computer Science
National University of Lesotho
w: 59105441 | e: setetemela.nul@gmail.com

Lecture Objectives

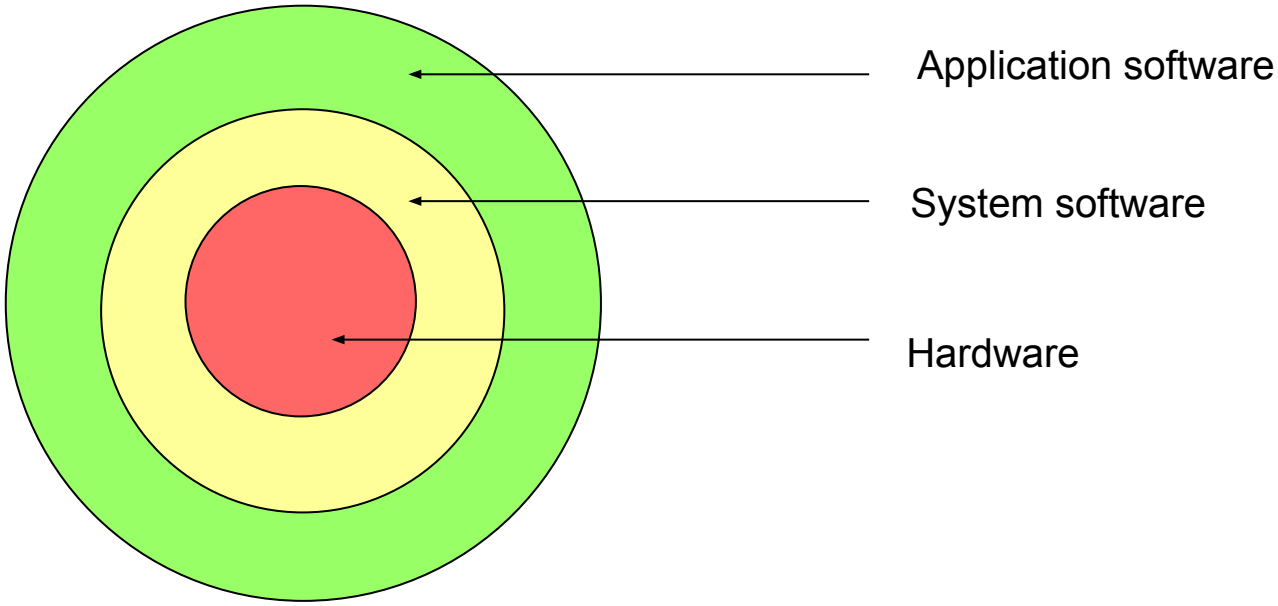
- Provide a system overview of a computer system
- Describe the structure and functions of the principal software components of a computer system
- Describe the structure and functions of the principal hardware components of a computer system

Lecture Outline

- Introduction
- Software components of a computer system
- Hardware components of a computer system
- Real stuff

INTRODUCTION:
Computer System Overview

Components of a Computer System



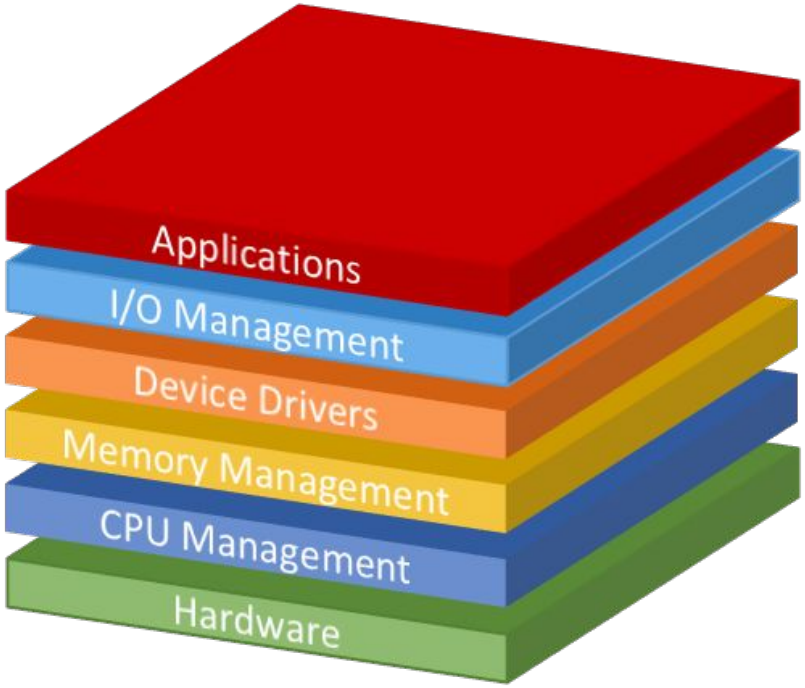
COMPUTER SYSTEM OVERVIEW:
Software Components

System Software

- Software designed to provide **platform** for **other computer software**
- Provides an **interface** between the computer system hardware and application software
- Many types of system software, but **Operating System** and **Compiler** are central to every modern computer system
- Other examples
  - Utility software
  - Device drivers
  - Shells

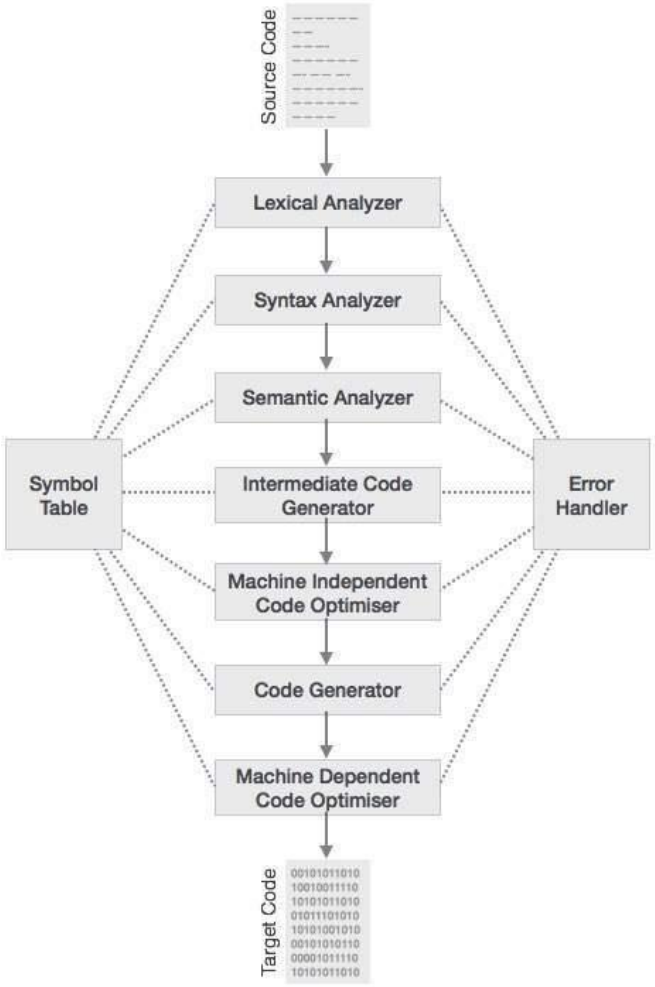
System Software – Operating System

- Provides a link between a user's application and the hardware
- Provides a variety of services and supervisory functions
- Some key OS functions include
  - Basic I/O operations
  - Storage and memory allocation
  - Protected sharing of the computer among multiple simultaneous applications
- Example operating systems
  - Linux
  - iOS
  - Android
  - Windows



System Software – Compiler System

- A compiler is a program that translates high-level
- language statements into assembly language statements
- There are several types of compilers
  - Cross compilers
  - Bootstrap compilers
  - Decompiler
  - Source to source compiler
  - Hardware compilers (synthesis)



Application Software

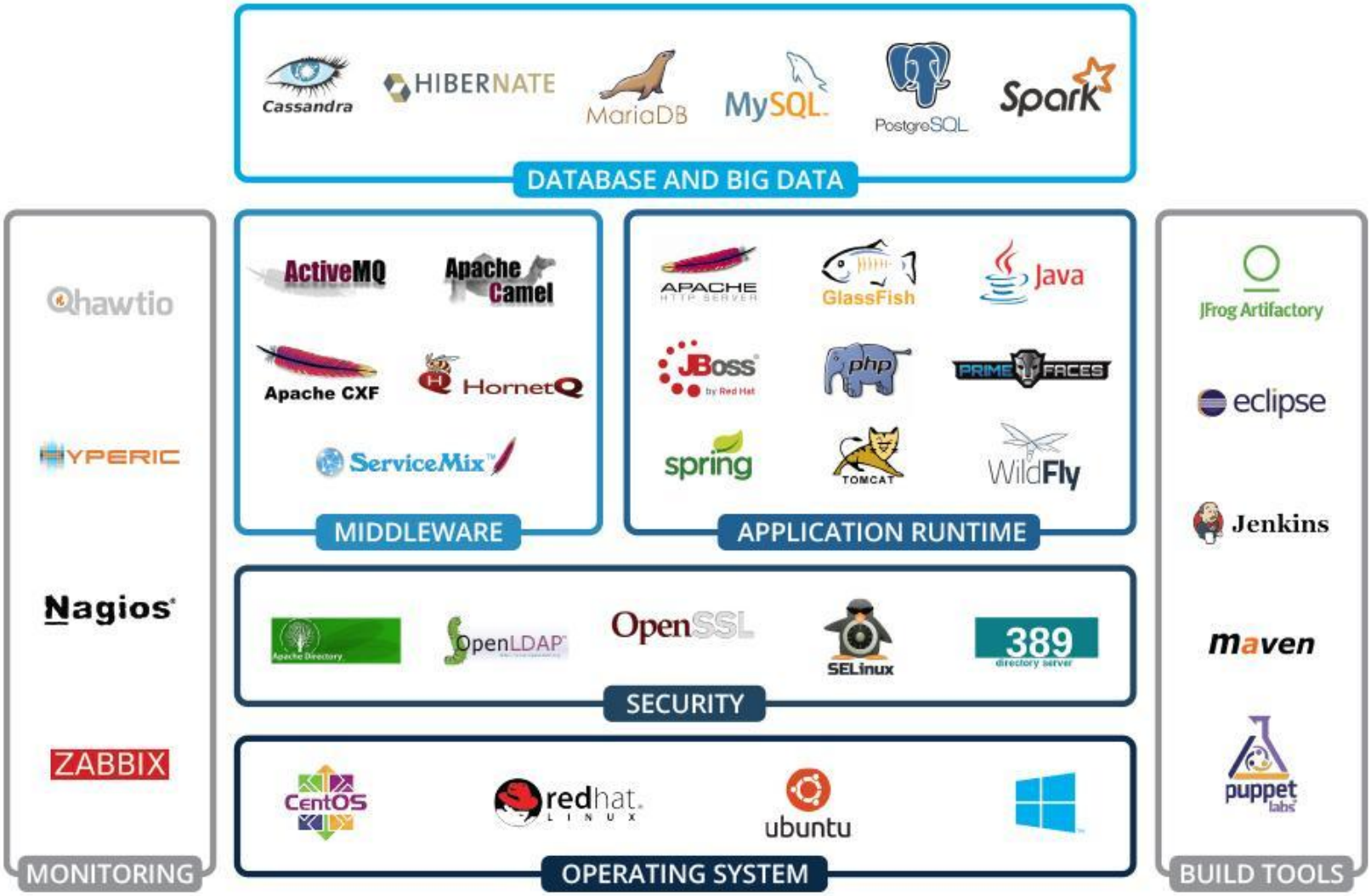
- Application software is software designed to perform a group of coordinated functions, tasks, or activities for the benefit of the user.
- Examples of an application include
  - Word processor
  - Spreadsheet
  - Accounting application
  - Web browser
  - Email client
  - Media player
  - File viewer
  - Photo editor.
- Applications may be bundled with the computer and its system software or published separately, and may be coded as **proprietary**, **open-source** or university projects

Application Software – Open Source

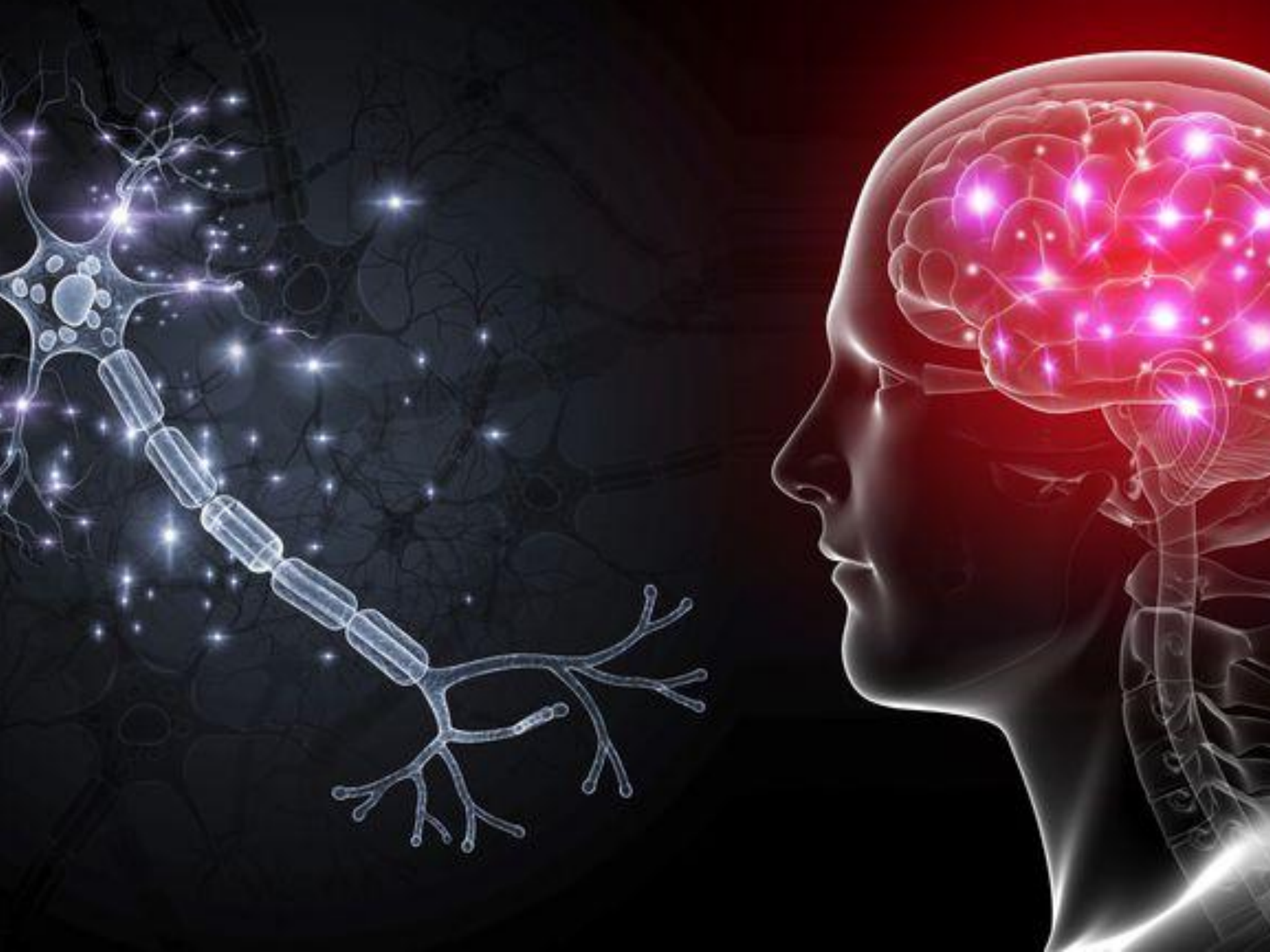
- Open-source software (OSS) is a type of computer software in which source code is released under a license in which the copyright holder grants users the rights to study, change, and distribute the software to anyone and for any purpose



Application Software – Open Source Stack

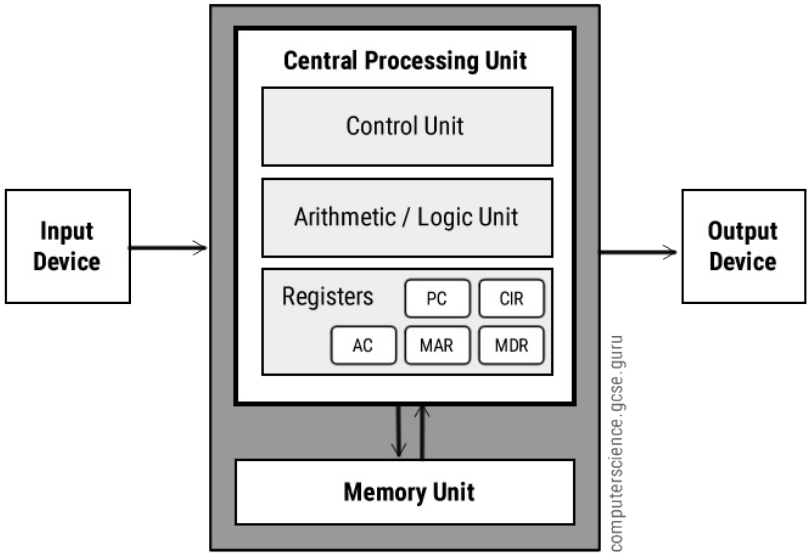


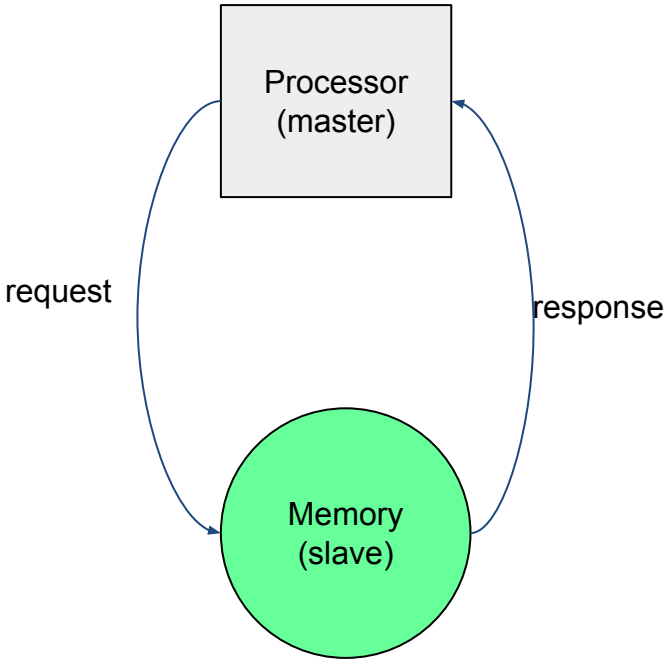
COMPUTER SYSTEM OVERVIEW:
Hardware Components



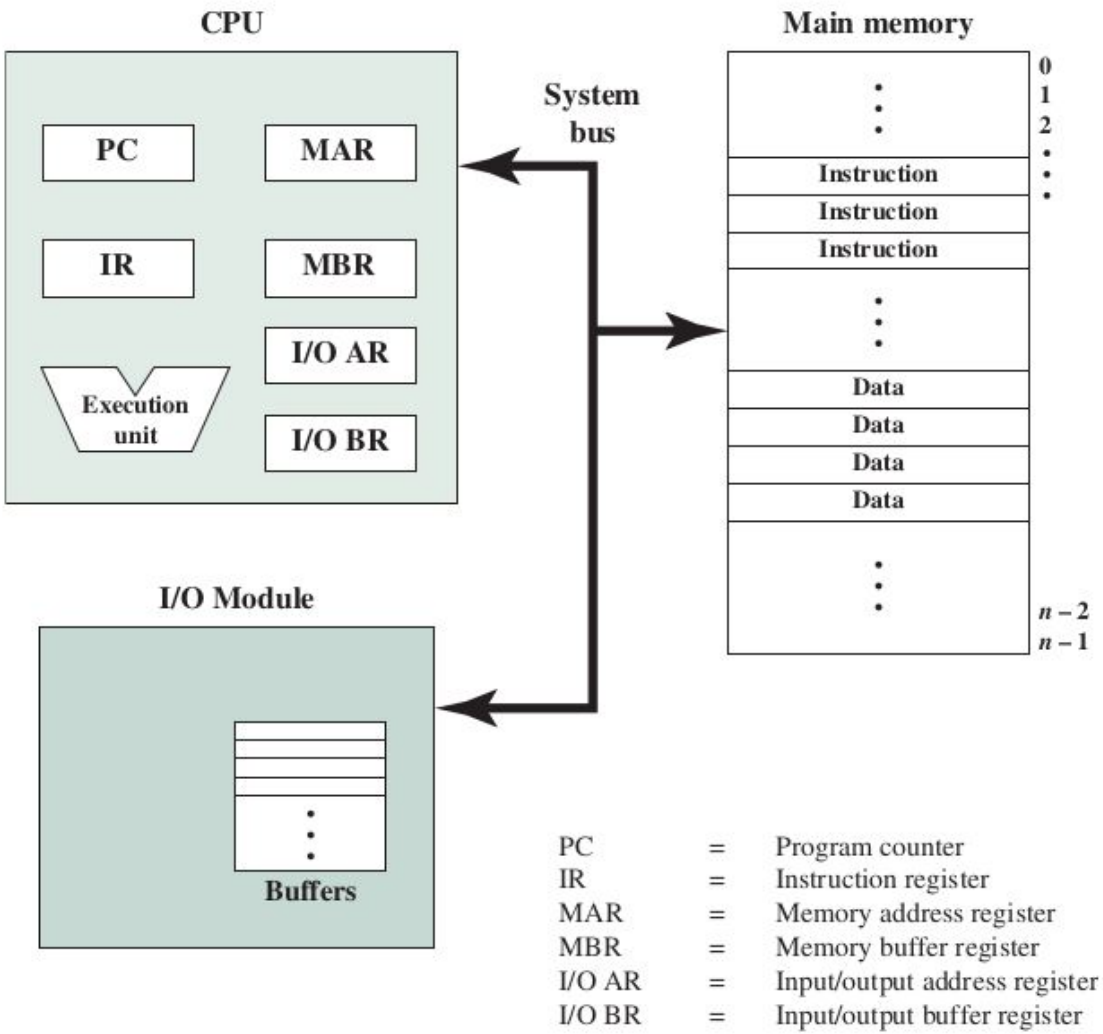


Hardware Components Overview





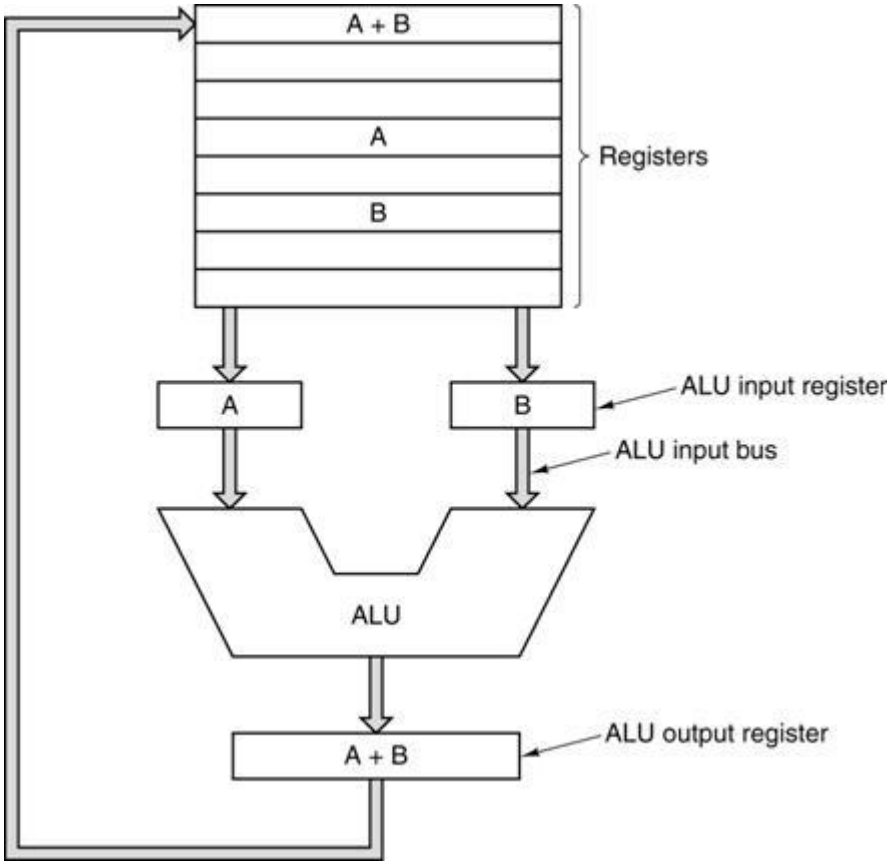
Hardware Components Overview



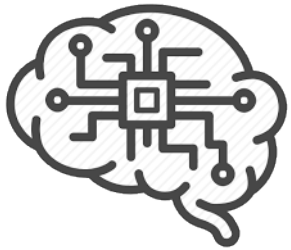
Processor

- Functions
  - Provides data **processing** capabilities
  - Data is essentially **numbers**
  - Processing entails carrying out **arithmetic**, **logic**, **control** and **I/O operations** as specified by the computer program
- Structure
  - Electronic circuitry
  - Principal components of the processor are the **datapath** and **control unit**
  - **Datapath** – includes the Arithmetic Logic Unit (ALU) – performs the arithmetic and logic operations, **Registers** – provides high speed access to operands for ALU operations and **multiplexers** for selecting between signals
  - **Control unit** – supervises fetching and execution of instructions by directing coordinated operations of the ALU, registers and other computer system components such as memory

Processor – Simple Datapath

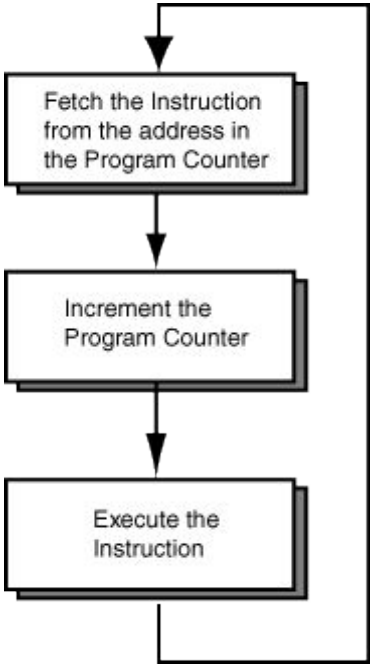


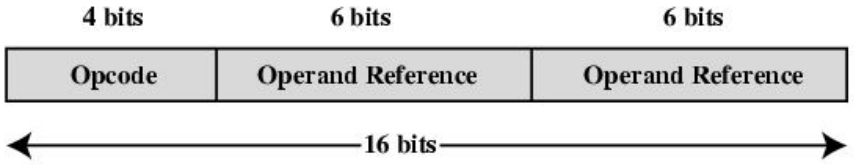
Processor – Control Unit

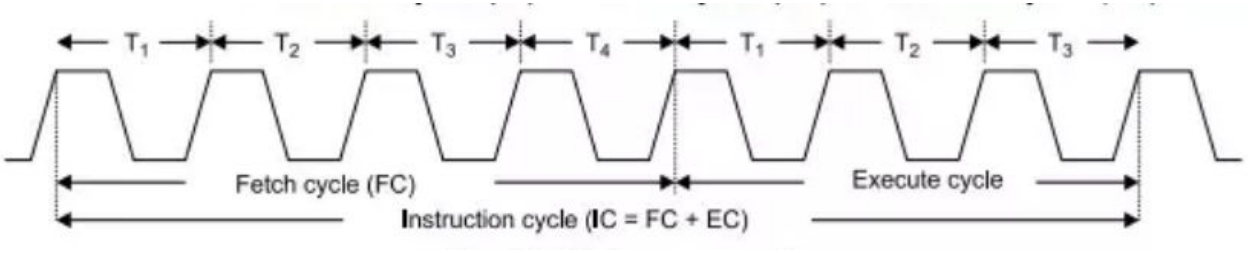


- The control unit (CU) is a component of a computer's central processing unit (CPU) that directs the operation of the processor.
  - It tells the computer's memory, arithmetic and logic unit and input and output devices how to respond to the instructions that have been sent to the processor.
- It directs the operation of the other units by providing timing and control signals.
  - Most computer resources are managed by the CU.
  - It directs the flow of data between the CPU and the other devices.
- In modern computer designs, the control unit is typically an internal part of the CPU with its overall role and operation unchanged since its introduction.

Processor – Control Unit: Machine Cycle

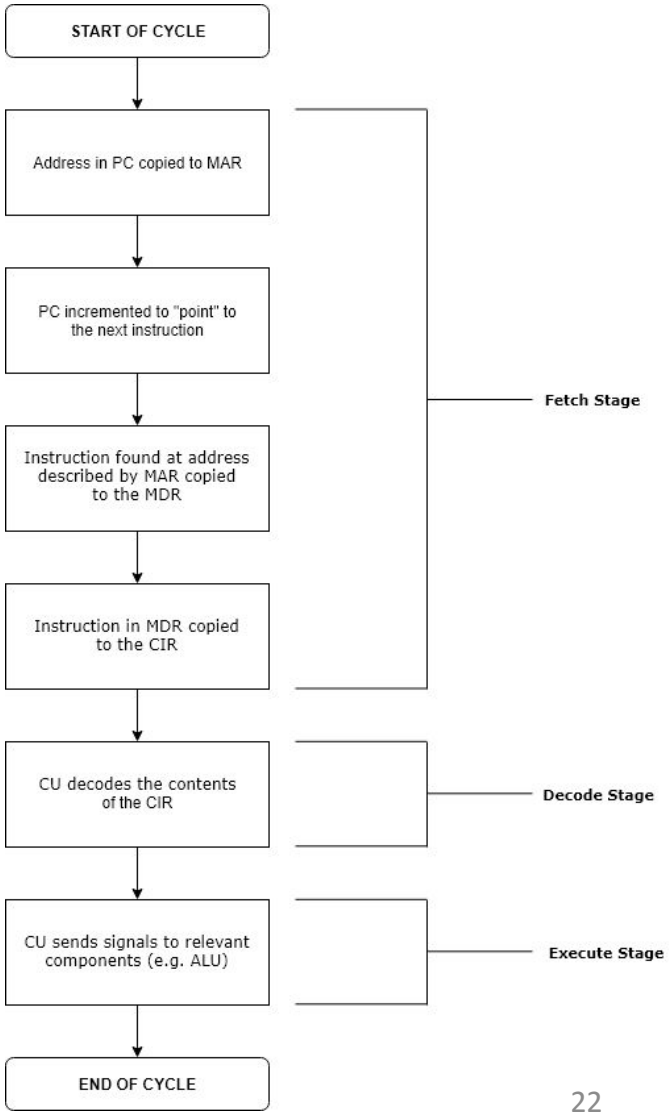






Processor – Control Unit: Machine Cycle in Detail

- The **program counter** (PC) is a special register that holds the address of the next instruction to be executed.
- During the **fetch stage**
  - The address stored in the PC is copied into the **memory address register** (MAR)
  - PC is incremented in order to "point" to the next instruction
  - The CPU then takes the instruction at MAR and copies it into the **memory data register** (MDR)
  - The instruction in the MDR is copied into the **current instruction register** (CIR)
- During the **decode stage**, the control unit (CU) decodes the instruction in the CIR
- During the **execute stage**
  - The CU sends the decoded information as a sequence of **control signals** to the relevant function units of the CPU to perform the actions required by the instruction
- In essence, the control unit is a **finite state machine (FSM) –** its operation involves a change from one **state** to another in response to some **inputs**!



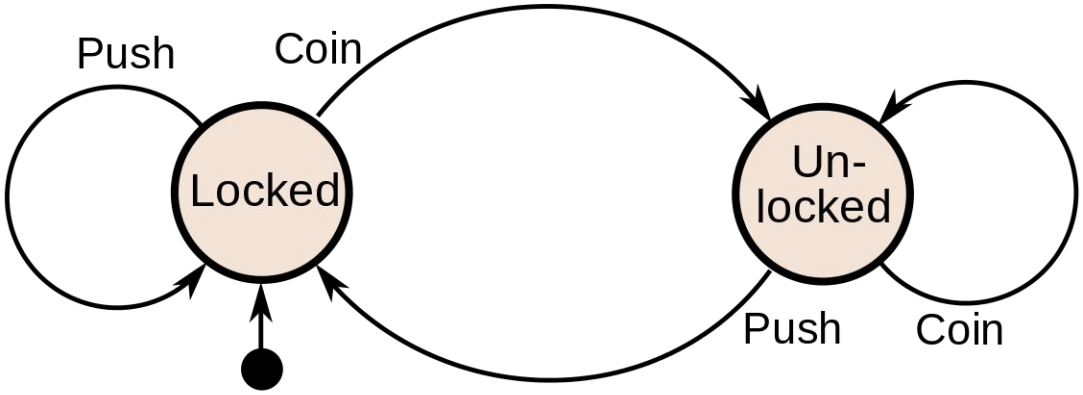
Processor – Control Unit: Finite State Machine Basics

- Considered as a **state machine**, the turnstile has **two possible states**:
  - **Locked** and **Unlocked**.
- There are two possible **inputs** that affect its state:
  - putting a coin in the slot (coin)
  - pushing the arm (push).
- In the **locked state**:
  - pushing on the arm has no effect;
  - Putting a coin in shifts the state from Locked to Unlocked.
- In the **unlocked state**:
  - putting additional coins in has no effect;
  - However, a customer pushing through the arms, shifts the state back to Locked.



Processor – Control Unit: Finite State Machine Basics

Current State	Input	Next State	Output
Locked	coin	Unlocked	Unlocks the turnstile so that the customer can push through.
	push	Locked	None
Unlocked	coin	Unlocked	None
	push	Locked	When the customer has pushed through, locks the turnstile.



Processor – Control Unit: Instruction Fetch Stage Activities

T1. CPU places address of next instruction on bus

ADDR.BUS ← MAR

T2. CPU generates the READ control signal

CTRL.BUS ← RD.CTRL (CPU)

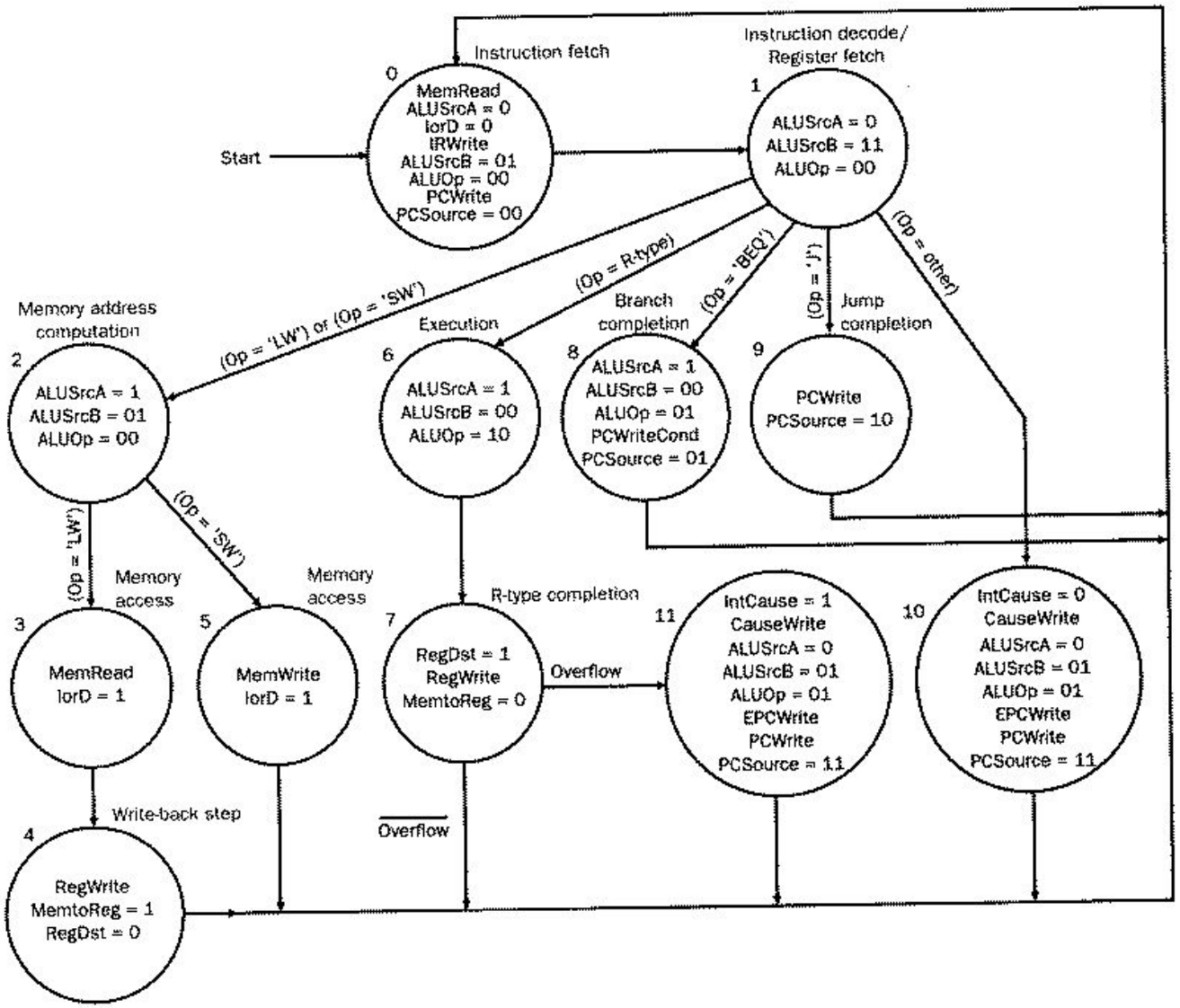
T3. Memory reads and places data (instruction) on the bus

DATA.BUS ← MDR (MEM)

T4. CPU reads in the data (instruction) placed on the bus by the memory in response to the CPU's request

CIR ← DATA.BUS (CPU)

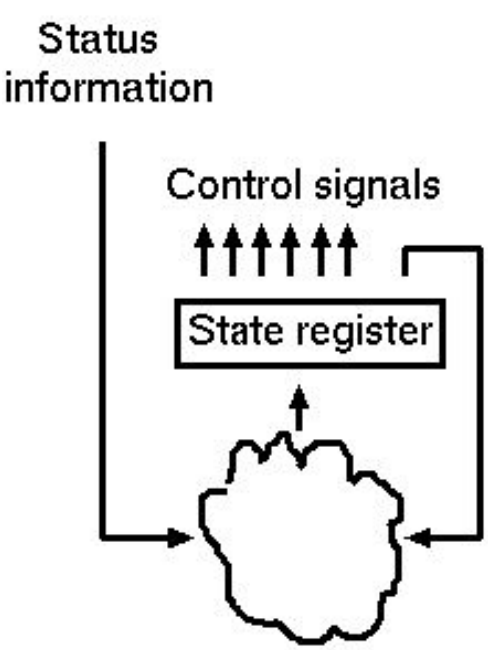
Example – MIPS Control Unit FSM



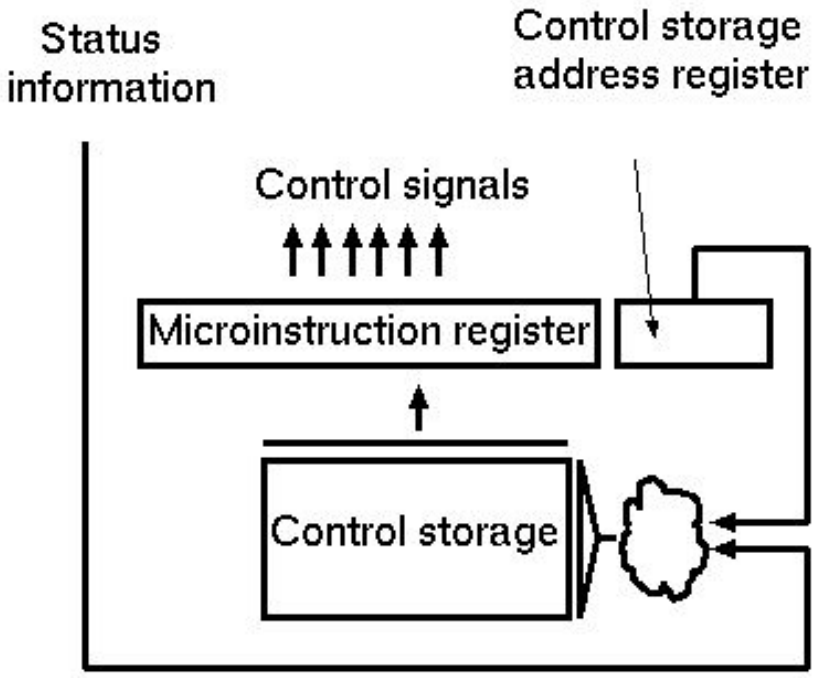
Processor – Control Unit: Implementation

- Microprogramming
  - The idea of **microprogramming** was introduced in 1951 as an intermediate level to execute computer program instructions.
  - Microprograms were organized as a sequence of **microinstructions** and stored in special **control memory**.
  - Main advantage is simplicity of microprogramming unit structure.
  - Outputs organized in microinstructions and can be easily replaced.
- Hardwired
  - Implemented through use of **combinational logic units.**
  - Generally faster than the microprogrammed designs but have high implementation complexity.
  - Design uses a fixed architecture—it **requires changes in the wiring if the instruction set is modified or change**d.
  - The hardwired approach has become less popular as computers have evolved.
  - Previously, control units for CPUs used ad-hoc logic, and they were difficult to design.

Processor – Control Unit: Microprogramming vs Hardwired



(a) Hardwired control



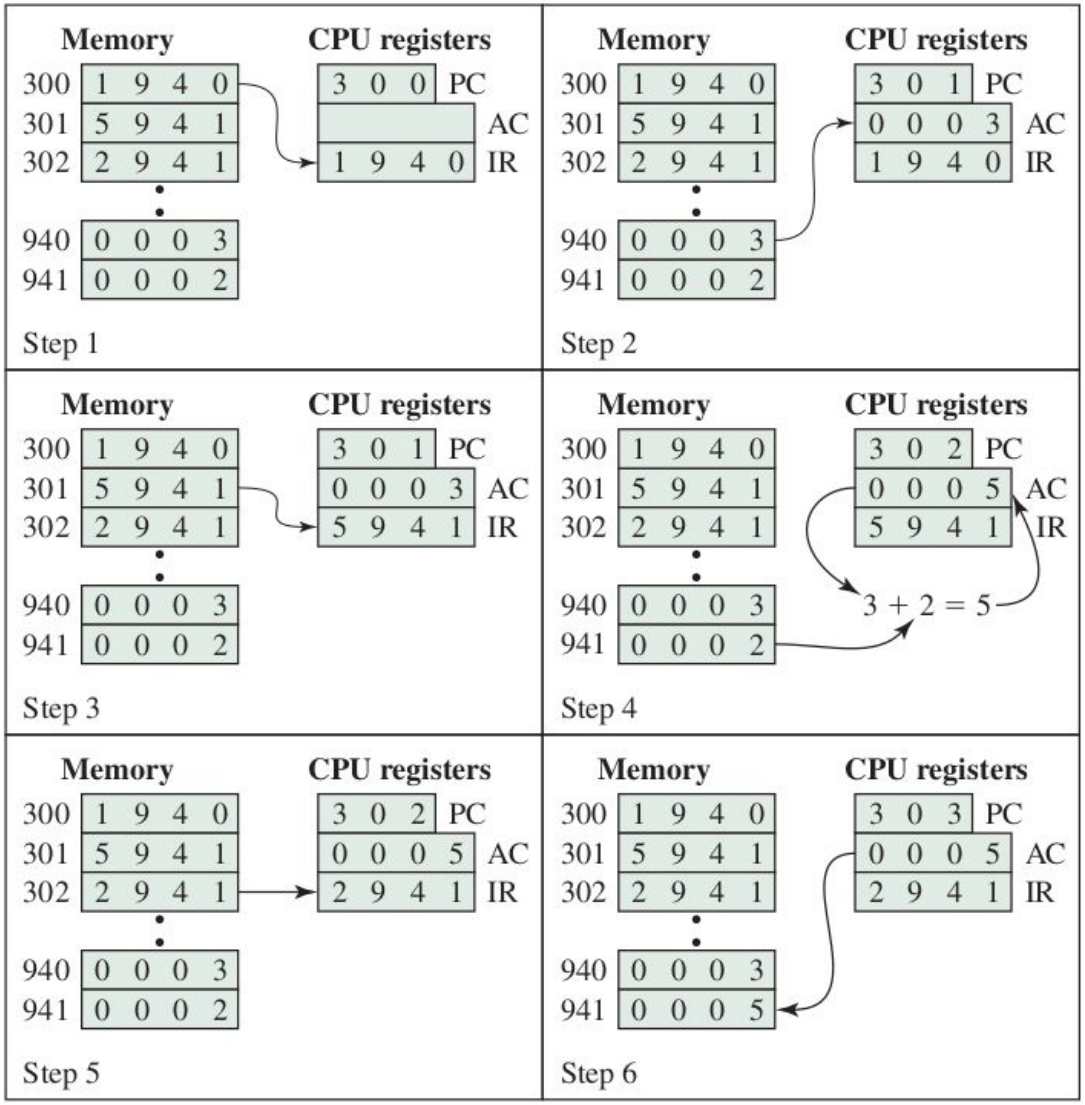
(b) Microprogrammed control

Example of Program Execution

0001 = Load AC from memory

0010 = Store AC to memory

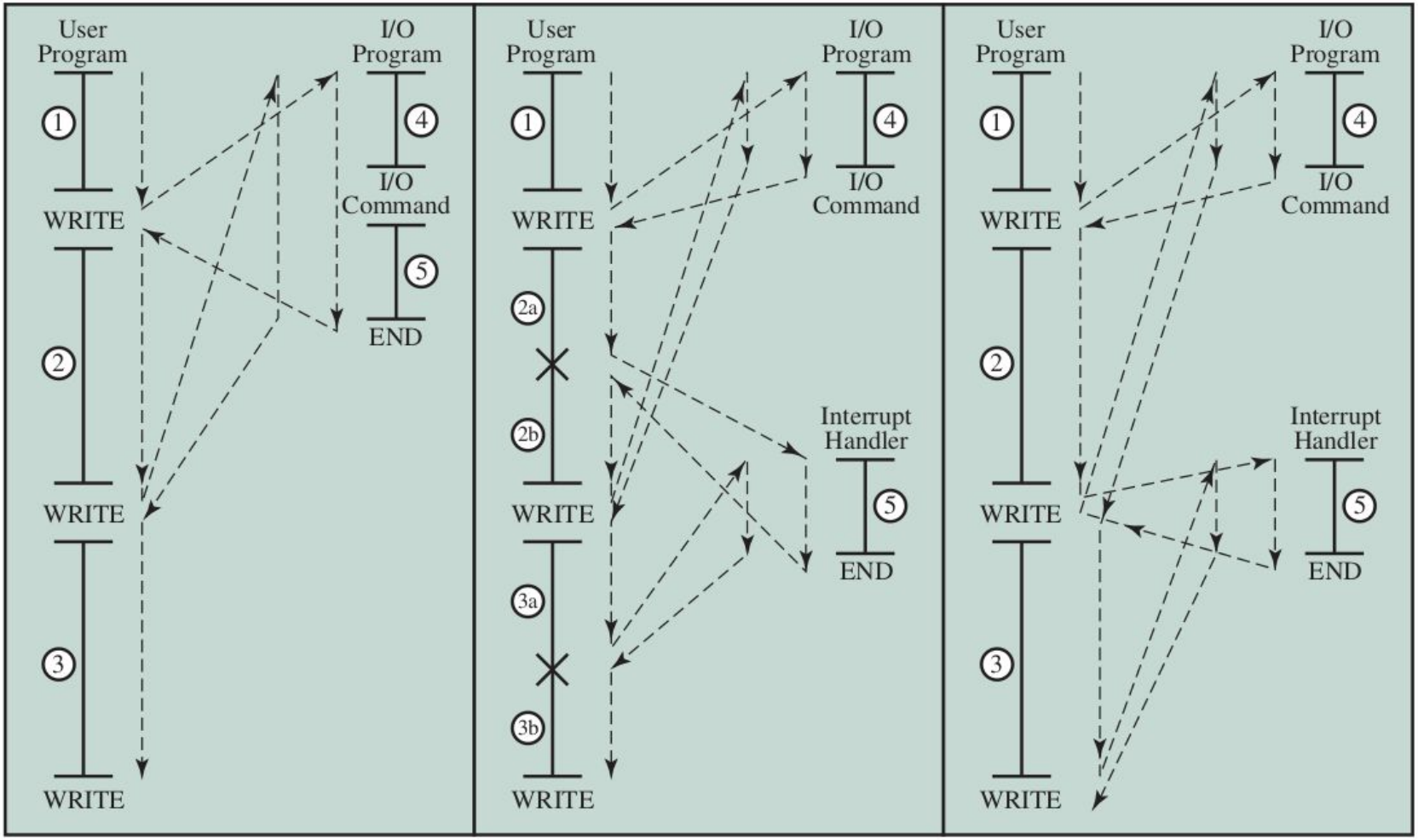
0101 = Add AC from memory



Interrupts

- **Purpose of Interrupts**:
  - Allows other modules (e.g., I/O, memory) to interrupt normal CPU processing.
  - Improves efficiency by avoiding CPU idle time during slow operations like I/O.
- **How Interrupts Work**:
  - Processor pauses its current operation.
  - Executes an **interrupt handler** to address the interrupt.
  - Resumes original program after handling the interrupt.
- **Interrupt Example**:
  - **Without Interrupts**: CPU waits during I/O operations, wasting cycles.
  - **With Interrupts**: CPU continues executing instructions while I/O is handled concurrently.
- **Key Steps in Interrupt Cycle**:
  - Processor detects an interrupt signal.
  - Suspends the current program and saves its context.
  - Executes the interrupt handler (part of the OS).
  - Resumes execution of the original program after handling.
- **Benefits**:
  - Maximizes CPU utilization.
  - Reduces idle time during I/O operations, improving overall system efficiency.

Interrupts and the Instruction Cycle

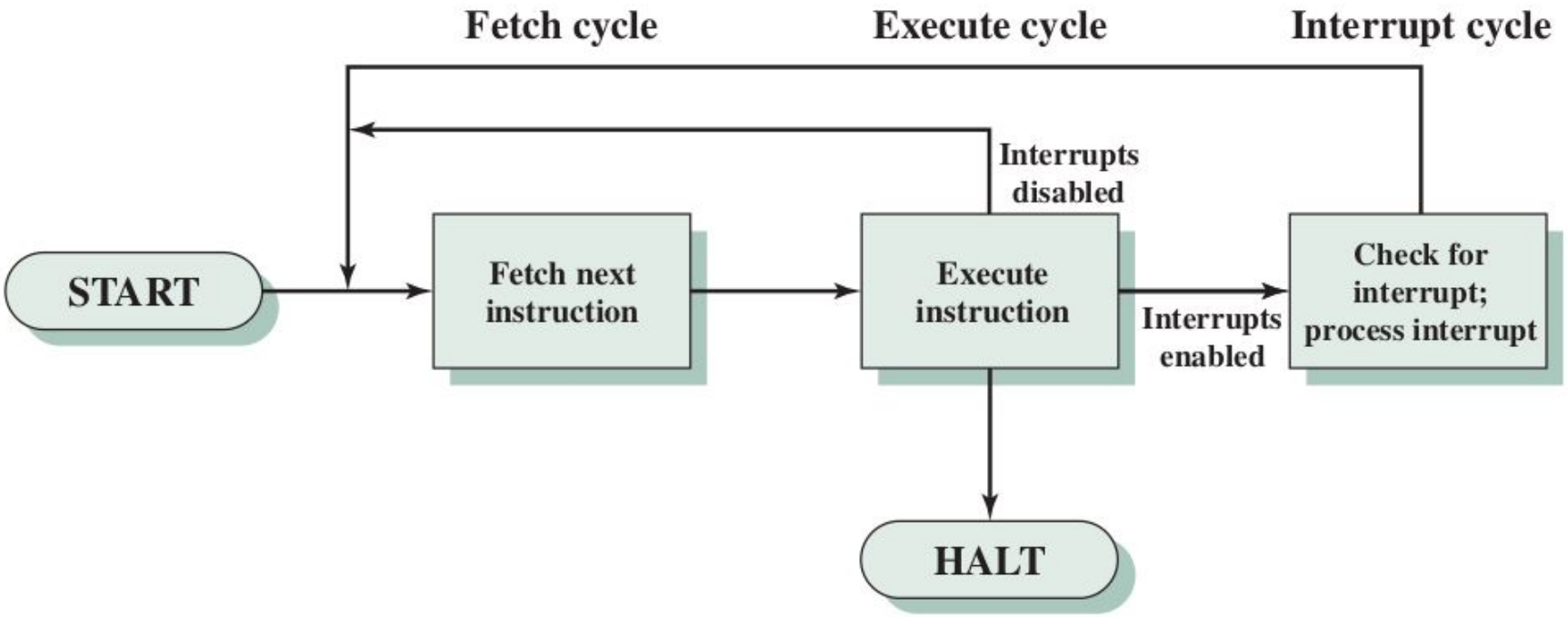


(a) No interrupts

(b) Interrupts; short I/O wait

(c) Interrupts; long I/O wait

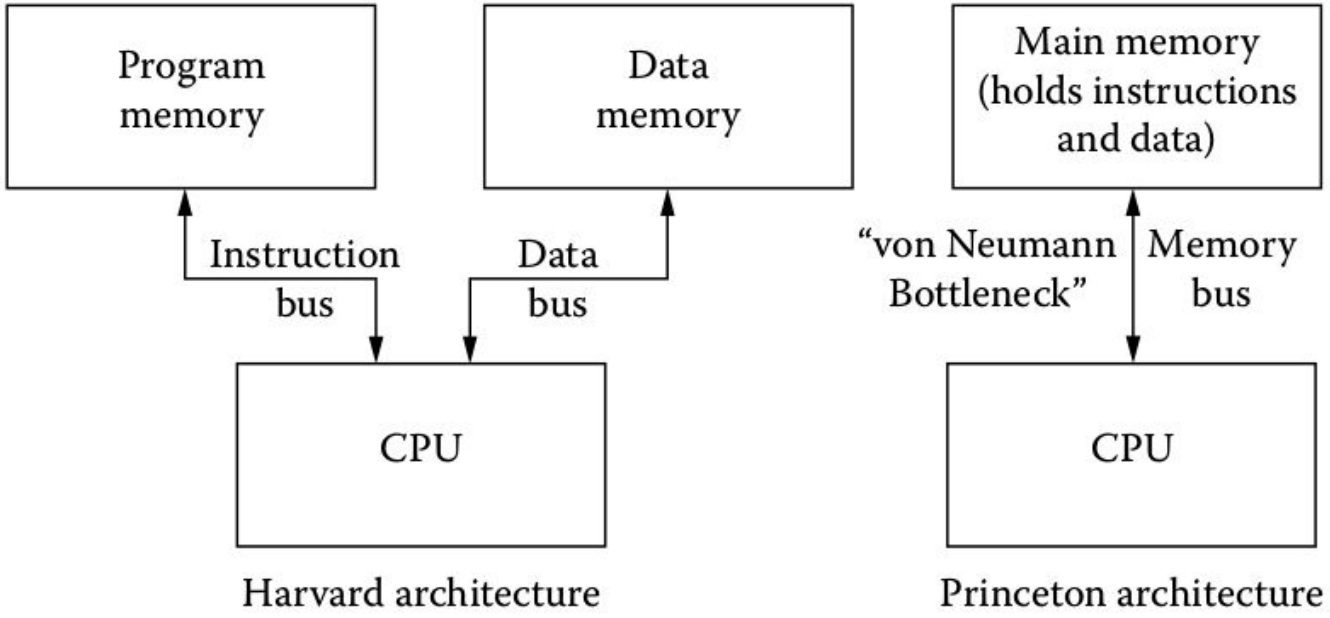
Interrupts and the Instruction Cycle



Memory

- **Categories of Memory:**
  - **Primary Storage**: Fast access, typically **RAM** (Random-Access Memory).
  - **Secondary Storage**: Slower, higher capacity storage (e.g., **Hard Drives**).
- **Types of Memory:**
  - **Volatile Memory**:
    - **DRAM (Dynamic RAM)**: Used as primary memory.
    - **SRAM (Static RAM)**: Used in CPU cache (faster but consumes more power).
  - **Non-Volatile Memory**:
    - **Flash Memory**: Used in secondary storage.
    - **ROM, PROM, EPROM, EEPROM**: Used for firmware (e.g., BIOS).
- **Other Key Concepts:**
  - **Virtual Memory**: Transfers contents between primary and secondary storage.

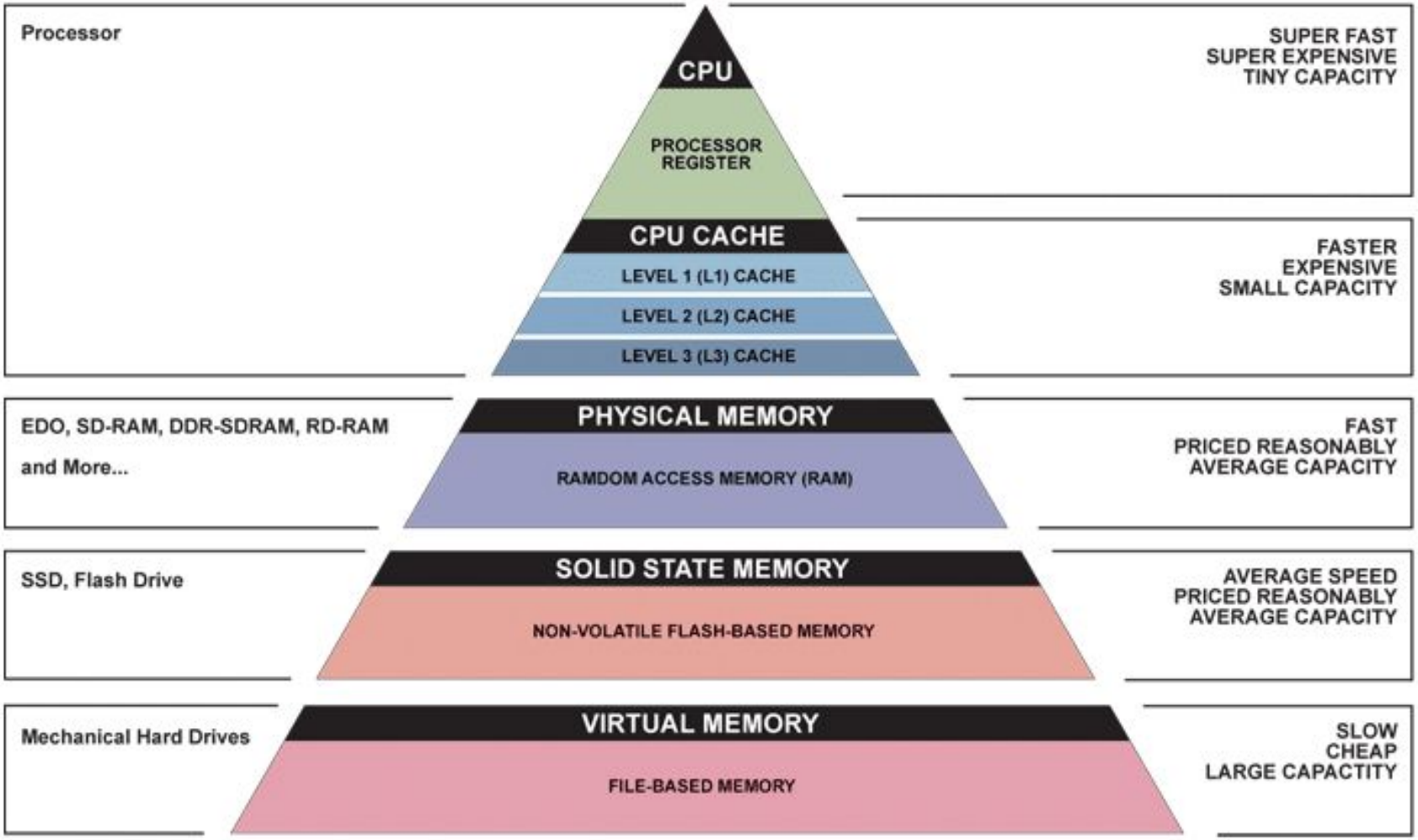
Memory - Von Neumann versus Harvard Architectures



- **Separate memories** and instructions for data and instructions
- Instructions may be fetched while data are being read or written
- **Increased performance** but **increased complexity** and implementation cost also

- Single path for accessing both instructions and data
- Both data and instructions stored in **same memory**
- Simple but suffers from the **"Von Neumann" bottleneck**

Memory Hierarchy



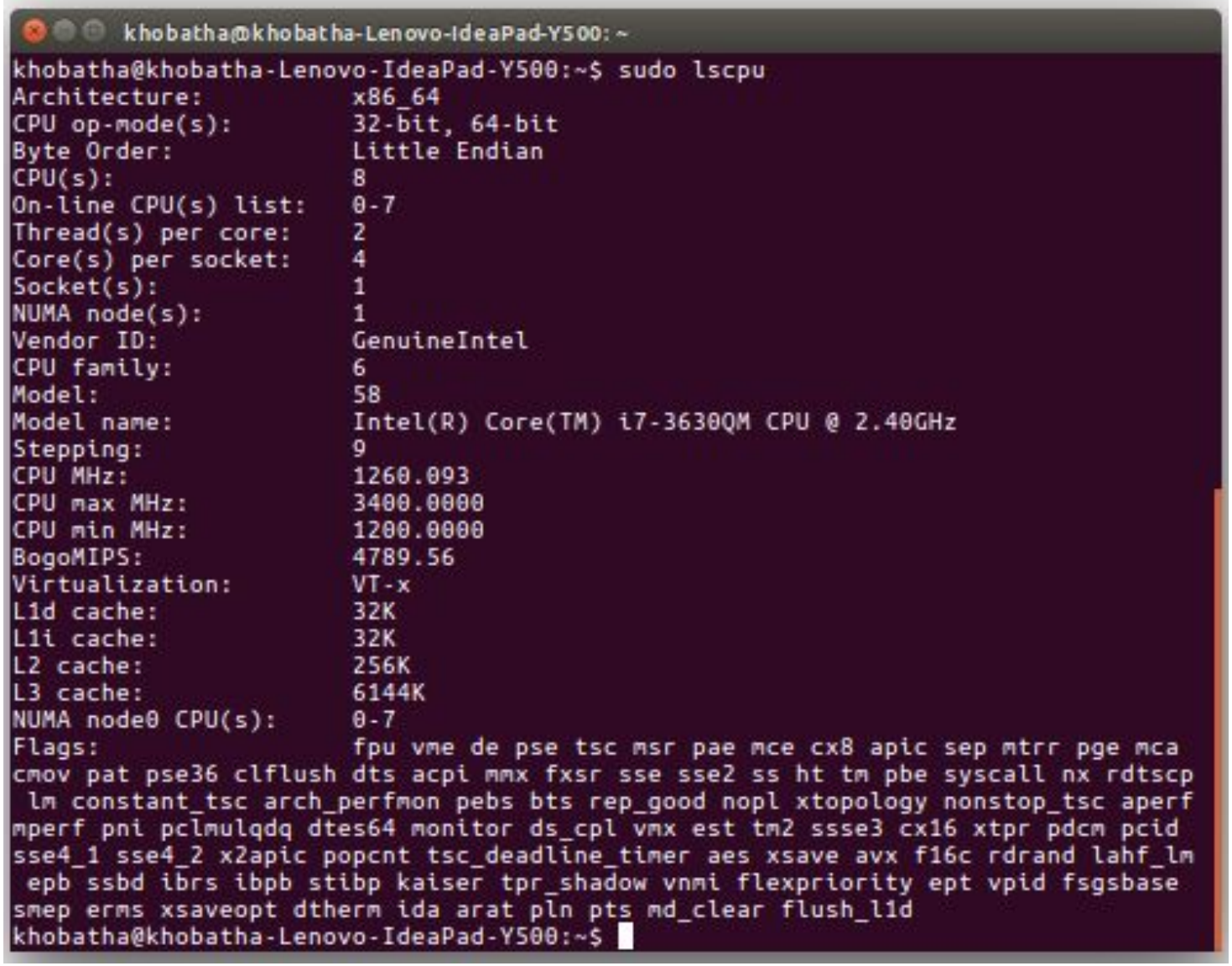
Memory Performance

- **Importance of Memory:**
  - Crucial for overall system performance.
  - Frequent interaction with CPU and I/O devices.
  - Poor memory design can bottleneck the entire system.
- **Key Performance Characteristics:**
  - **Size:**
    - **Main Memory**: Should be large enough to keep most active programs and data accessible by the CPU without costly disk accesses.
    - **Secondary Storage**: Must provide sufficient capacity for long-term storage of programs and data.
    - **Memory Units**: Byte (B), Kilobyte (KB), Megabyte (MB), Gigabyte (GB), Terabyte (TB), Petabyte (PB).
  - **Speed:**
    - **Main Memory**: Needs to match the speed of CPU and I/O devices to avoid delays.
    - **Disk Drives**: Should be fast enough to load large applications or data frequently.
    - **Speed Metrics**: Cycle time, access time (seek time + latency + transfer time), memory bandwidth.

Memory Size Measurement

Unit of Storage	Abbreviation	Approximate Size	Exact Size
Kilobyte	KB	1 thousand (10^3) bytes	1024 (2^{10}) bytes
Megabyte	MB	1 million (10^6) bytes	1,048,576 (2^{20}) bytes
Gigabyte	GB	1 billion (10^9) bytes	1,073,741,824 (2^{30}) bytes
Terabyte	TB	1 trillion (10^{12}) bytes	1.0995×10^{12} (2^{40}) bytes

Example- Intel Core i7 Processor Information



I/O



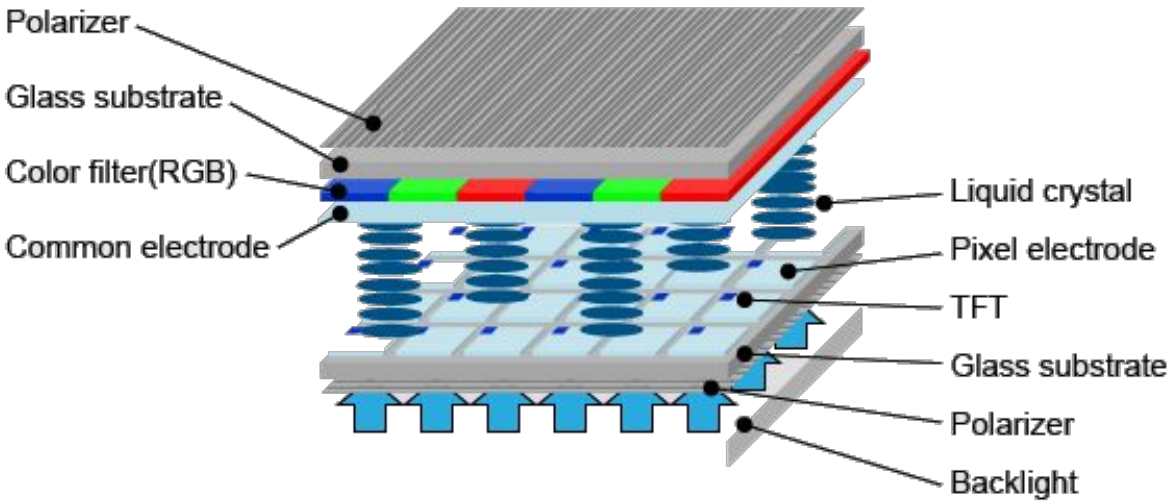
- **Purpose of I/O:**
  - To move data and programs into and out of the computer system.
- **Types of I/O Devices**
  - **Input Devices**
    - Allow humans (or other systems) to communicate data to the computer.
    - Examples: **Keyboard, Mouse, Scanner, Voice Assistants (e.g Amazon Alexa, Google Assistant)**
  - **Output Devices**:
    - Present data from the computer to humans or other systems.
    - Examples: **Monitor, 3D Printers, Speakers, AV/VR headsets**
  - **Dual-Function Devices**
    - **Communication Devices**:
      - Perform both input and output operations.
      - Examples: **Modems, Network Cards, AV/VR devices**

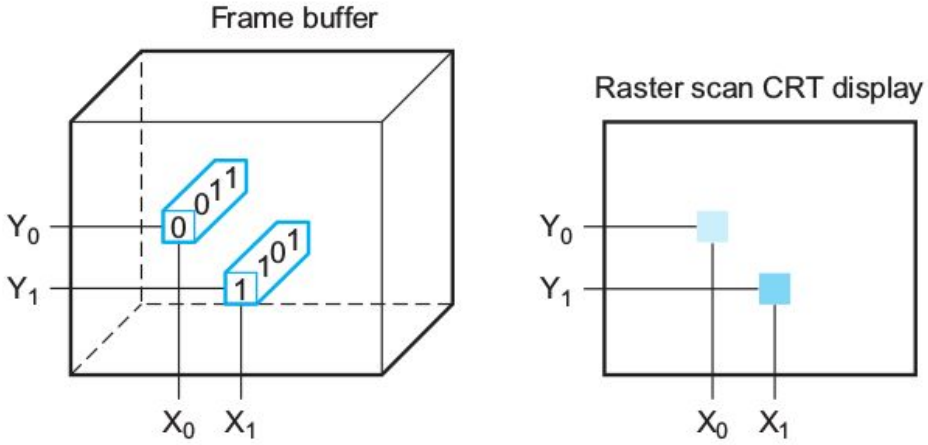


I/O – LCD Display

- **Pixels and Sub-pixels**:
  - **Pixels**: Each pixel is composed of three sub-pixels (Red, Green, Blue - RGB).
  - **Full-HD Example**: In Full-HD resolution (1080 x 1920), there are over 6 million sub-pixels (1,080 x 1,920 x 3 = 6,220,800).
- **Thin Film Transistor (TFT)**:
  - **Purpose**: A **TFT** is required to control each sub-pixel, acting as a valve that applies voltage to liquid crystals, determining the light passing through.
  - **Function**: Controls voltage for individual sub-pixels to modulate the liquid crystals.
- **TFT LCD Structure**:
  - **Glass Substrate**:
    - One layer has **TFTs** and **transparent pixel electrodes**.
    - Another glass layer contains a **color filter** (RGB) and **transparent counter electrodes**.
  - **Liquid Crystal Layer**: Positioned between these two glass substrates.
  - **Polarizers**: Placed on the outer sides of each glass substrate.
  - **Backlight**: Located behind the glass, providing light that passes through the liquid crystals.
- **Display Operation**:
  - **Voltage Application**: Changing the voltage to the liquid crystals alters the transmittance of the panel, affecting the amount of light passing through.
  - **Light Transmission**: The interaction between the liquid crystals and polarizing plates controls the light from the backlight, allowing full-color image production.
- **Graphics Support**
  - **Frame Buffer**: Stores the bitmap of the display.
  - **Bit Pattern**: The bit pattern per pixel is read out from the frame buffer at the display refresh rate, updating the image.

I/O – LCD Display





Bus Interconnection Overview

- **Bus as Traditional Interconnection**:
  - Dominated computer system design for decades.
  - Gradually replaced by point-to-point interconnections in general-purpose computers.
  - Still used in **embedded systems** like **microcontrollers**.
- **Bus Characteristics**:
  - **Communication Pathway**: Connects multiple devices.
  - **Shared Transmission Medium**: Only one device can transmit at a time.
- **Bus Structure**:
  - Composed of multiple lines transmitting binary signals.
  - Allows parallel transmission (e.g., 8-bit data over 8 lines).
- **System Bus**:
  - Connects major components (e.g., **processor**, **memory**, **I/O**).
  - Consists of **data lines**, **address lines**, and **control lines**.
- **Key Functional Groups**:
  - **Data Lines**: Move data among system modules; width affects performance.
  - **Address Lines**: Identify the source/destination of data; width determines system memory capacity.
  - **Control Lines**: Manage access to data and address lines, transmitting commands (e.g., memory read/write, I/O operations).
- **Control Signals**:
  - Include **memory read/write**, **I/O read/write**, **interrupt handling**, **bus request/grant**, **clock**, and **reset**.
- **Bus Operation**:
  - Modules must obtain control of the bus to transfer or request data.

Bus Interconnection Overview

